

Documentación Técnica — Core del Sistema Task Manager

Lógica de Negocio, Rutas de Archivo y Código Fuente

ÍNDICE

1. Autenticación — Middleware SupabaseAuth
 2. Creación y Validación de Tareas — TaskController
 3. Cálculo de Carga de Trabajo — WorkloadController
 4. Sugerencias de Asignación — WorkloadController
 5. Gestión de Equipos y Miembros — TeamController
 6. Gestión de Usuarios y Skills — ProfileController
-

1. AUTENTICACIÓN

Nombre: Middleware de Autenticación Supabase **Ruta:**

[backend/app/Http/Middleware/SupabaseAuth.php](#) **Se ejecuta en:** cada request antes de llegar a cualquier Controller

Código completo

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Http;

class SupabaseAuth
{
    public function handle(Request $request, Closure $next)
    {
        $token = $request->bearerToken();

        if (!$token) {
            return response()->json(['error' => 'Token no proporcionado'],
401);
        }

        try {
```

```

        $supabaseUrl = env('SUPABASE_URL');
        $response = Http::withHeaders([
            'Authorization' => 'Bearer ' . $token,
            'apikey' => env('SUPABASE_ANON_KEY'),
        ])->get($supabaseUrl . '/auth/v1/user');

        if ($response->failed()) {
            return response()->json(['error' => 'Token inválido'],
401);
        }

        $user = $response->json();
        $request->merge(['user_id' => $user['id']]);
    } catch (\Exception $e) {
        return response()->json(['error' => 'Error de autenticación'],
401);
    }

    return $next($request);
}
}

```

Explicación línea a línea

Línea	Código	Qué hace
<code>\$token = \$request->bearerToken()</code>	Extrae el JWT del header <code>Authorization: Bearer <token></code>	
<code>if (!\$token)</code>	Si no hay token → responde 401 de inmediato, no continúa	
<code>Http::withHeaders([...])->get(...)</code>	Llama al endpoint de Supabase <code>/auth/v1/user</code> con el token para validarlo	
<code>if (\$response->failed())</code>	Si Supabase dice que el token es inválido → 401	
<code>\$request->merge(['user_id' => \$user['id']])</code>	Injecta el UUID del usuario en el request para que los Controllers lo usen	
<code>return \$next(\$request)</code>	Continúa hacia el Controller si todo fue bien	

Flujo completo

Cliente → HTTP Request con Bearer token

- └ Sin token → 401 "Token no proporcionado"
- └ Token malo → Supabase valida → 401 "Token inválido"

```
| Token OK    → $request->user_id = UUID del usuario  
                → continúa al Controller
```

Tablas de DB involucradas: ninguna. La validación es 100% remota via HTTP a Supabase Auth.

2. CREACIÓN Y VALIDACIÓN DE TAREAS

Nombre: Controlador de Tareas **Ruta:** `backend/app/Http/Controllers/TaskController.php`

Endpoints cubiertos:

- `POST /api/tasks` — crear tarea
- `PUT /api/tasks/{id}` — actualizar tarea

Código completo — `store()` (crear tarea)

```
public function store(Request $request)
{
    $request->validate([
        'title'           => 'required|string',
        'description'      => 'nullable|string',
        'priority'         => 'required|in:low,medium,high',
        'estimated_hours' => 'required|numeric|min:0.5|max:40',
        'due_date'         => 'nullable|date',
        'team_id'          => 'required|exists:teams,id',
        'assigned_to'      => 'required|exists:profiles,id',
        'status'           => 'in:pending,in_progress,completed',
        'project_id'       => 'nullable|exists:projects,id',
    ]);

    $memberExists = TeamMember::where('team_id', $request->team_id)
        ->where('user_id', $request->assigned_to)
        ->exists();

    if (!$memberExists) {
        return response()->json([
            'errors' => ['assigned_to' => ['El usuario asignado no
pertenece al equipo seleccionado']]
        ], 422);
    }

    $task = Task::create([
        'title'           => $request->title,
        'description'      => $request->description,
        'priority'         => $request->priority,
        'estimated_hours' => $request->estimated_hours,
        'due_date'         => $request->due_date,
        'team_id'          => $request->team_id,
```

```

        'assigned_to'    => $request->assigned_to,
        'status'         => $request->status ?? 'pending',
        'user_id'        => $request->user_id,
        'created_by'     => $request->user_id,
        'project_id'     => $request->project_id,
    ]);

    return response()->json($task->load(['assignee', 'team', 'project']),
201);
}

```

Explicación

Bloque 1 — Validación de tipos y existencia de FK

```

$request->validate([
    'team_id'    => 'required|exists:teams,id',    // consulta SELECT en
    tabla teams
    'assigned_to'=> 'required|exists:profiles,id', // consulta SELECT en
    tabla profiles
    'project_id' => 'nullable|exists:projects,id', // consulta SELECT en
    tabla projects
    'priority'   => 'required|in:low,medium,high', // enum controlado
    'estimated_hours' => 'required|numeric|min:0.5|max:40',
]);

```

Laravel ejecuta `SELECT COUNT(*) FROM teams WHERE id = ?` para cada campo `exists:`. Si el ID no existe en la tabla referenciada, devuelve 422 automáticamente.

Bloque 2 — Validación cross-table (el core de la asignación)

```

$memberExists = TeamMember::where('team_id', $request->team_id)
->where('user_id', $request->assigned_to)
->exists();

```

Query generada:

```

SELECT COUNT(*) > 0
FROM team_members
WHERE team_id = '{team_id recibido}'
AND user_id = '{assigned_to recibido}'

```

Esta es la validación de negocio principal: verifica que la persona a asignar pertenezca al equipo. Navega la tabla `team_members` que es la junction table entre `teams` y `profiles`.

Bloque 3 — Creación con trazabilidad dual

```

$task = Task::create([
    'user_id' => $request->user_id,    // UUID de quien hizo la request
    (inyectado por SupabaseAuth)
    'created_by' => $request->user_id,    // quién creó (FK → profiles)
    'assigned_to' => $request->assigned_to, // a quién se asigna (FK →
    profiles)
    'project_id' => $request->project_id, // a qué proyecto (FK → projects,
    nullable)
]);

```

La tarea guarda dos FKs diferentes a **profiles**: **created_by** y **assigned_to**. Pueden ser la misma persona o personas distintas.

Código completo — **update()** (actualizar tarea)

```

public function update(Request $request, $id)
{
    $task = Task::findOrFail($id);

    $request->validate([
        'title' => 'sometimes|string',
        'description' => 'nullable|string',
        'priority' => 'sometimes|in:low,medium,high',
        'estimated_hours' => 'sometimes|numeric|min:0.5|max:40',
        'due_date' => 'nullable|date',
        'team_id' => 'sometimes|exists:teams,id',
        'assigned_to' => 'sometimes|exists:profiles,id',
        'status' => 'sometimes|in:pending,in_progress,completed',
        'project_id' => 'nullable|exists:projects,id',
    ]);

    $teamId = $request->team_id ?? $task->team_id;
    $assignedTo = $request->assigned_to ?? $task->assigned_to;

    if ($request->has('assigned_to') || $request->has('team_id')) {
        $memberExists = TeamMember::where('team_id', $teamId)
            ->where('user_id', $assignedTo)
            ->exists();

        if (!$memberExists) {
            return response()->json([
                'errors' => ['assigned_to' => ['El usuario asignado no
                pertenece al equipo seleccionado']]
            ], 422);
        }
    }

    $task->update($request->only([
        'title', 'description', 'priority', 'estimated_hours',

```

```

        'due_date', 'team_id', 'assigned_to', 'status', 'project_id',
    ]));

    return response()->json($task->load(['assignee', 'team', 'project']));
}

```

Explicación del update

Lógica de resolución de valores:

```

$teamId      = $request->team_id    ?? $task->team_id;    // si no viene en
el request, usa el actual
$assignedTo = $request->assigned_to ?? $task->assigned_to; // idem

```

Esto permite actualizaciones parciales: si solo se cambia el **status**, no re-valida la asignación.

Re-validación condicional:

```

if ($request->has('assigned_to') || $request->has('team_id')) {
    // Solo re-verifica si se intentó cambiar el asignado o el equipo
    $memberExists = TeamMember::where('team_id', $teamId)
        ->where('user_id', $assignedTo)
        ->exists();
}

```

La validación cross-table se re-ejecuta únicamente cuando es relevante: cuando se modifica el equipo o el asignado.

Tablas involucradas:

```

team_members    ← validación cross-table
profiles        ← verifica que assigned_to exista
teams           ← verifica que team_id exista
projects        ← verifica que project_id exista (si viene)
tasks           ← actualización final

```

3. CÁLCULO DE CARGA DE TRABAJO

Nombre: Workload Index **Ruta:** `backend/app/Http/Controllers/WorkloadController.php` — método `index()` **Endpoint:** `GET /api/workload?team_id=X&start_date=Y&end_date=Z`

Código completo — `index()`

```

public function index(Request $request)
{
    $request->validate([
        'team_id'      => 'required|exists:teams,id',
        'start_date'   => 'nullable|date',
        'end_date'     => 'nullable|date|after_or_equal:start_date',
    ]);

    $members = TeamMember::where('team_id', $request->team_id)
        ->with('profile')
        ->get();

    // Una sola query para todos los miembros en lugar de N queries en el
    loop
    $taskStats = Task::where('team_id', $request->team_id)
        ->where('status', '!=', 'completed')
        ->when(
            $request->start_date && $request->end_date,
            fn ($q) => $q->whereBetween('due_date', [$request->start_date,
            $request->end_date])
        )
        ->selectRaw('assigned_to, count(*) as total_tasks,
        coalesce(sum(estimated_hours), 0) as total_hours')
        ->groupBy('assigned_to')
        ->get()
        ->keyBy('assigned_to');

    $membersData = $members->map(function ($member) use ($taskStats) {
        $stats      = $taskStats->get($member->user_id);
        $hours      = $stats ? (float) $stats->total_hours : 0;
        $tasks      = $stats ? (int) $stats->total_tasks : 0;
        $capacity   = (int) ($member->profile->weekly_hours_capacity ?? 40);
        $loadPct    = $capacity > 0 ? round($hours / $capacity * 100) : 100;

        $workloadLevel = match (true) {
            $loadPct < 40 => 'low',
            $loadPct <= 70 => 'medium',
            default      => 'high',
        };

        return [
            'id'          => $member->user_id,
            'name'        => $member->profile->first_name . ' ' .
            $member->profile->last_name,
            'email'       => $member->profile->email,
            'tasks_count' => $tasks,
            'assigned_hours' => $hours,
            'capacity'    => $capacity,
            'workload_pct' => $loadPct,
            'workload_level' => $workloadLevel,
        ];
    });
}

```

```

$totalHours = $membersData->sum('assigned_hours');
$totalTasks = $membersData->sum('tasks_count');
$count      = $membersData->count();

$suggested = $membersData->isEmpty()
    ? $membersData->sortBy('workload_pct')->first()
    : null;

return response()->json([
    'summary' => [
        'total_tasks'    => $totalTasks,
        'total_hours'    => $totalHours,
        'average_hours' => $count > 0 ? round($totalHours / $count, 2)
    ],
    'members'           => $membersData->values(),
    'suggested_assignee' => $suggested,
]);
}

```

Explicación bloque a bloque

Bloque 1 — Obtener miembros del equipo

```

$members = TeamMember::where('team_id', $request->team_id)
    ->with('profile')
    ->get();

```

Query generada:

```

SELECT * FROM team_members WHERE team_id = '{id}'
-- + eager loading:
SELECT * FROM profiles WHERE id IN (user_id1, user_id2, ...)

```

Navega `team_members` y trae en una segunda query todos los `profiles` de esos miembros (eager loading evita N+1).

Bloque 2 — Agregar horas por miembro (1 sola query para todos)

```

$taskStats = Task::where('team_id', $request->team_id)
    ->where('status', '!=', 'completed')
    ->when(
        $request->start_date && $request->end_date,
        fn ($q) => $q->whereBetween('due_date', [$request->start_date,
            $request->end_date])
    )

```



```

->selectRaw('assigned_to, count(*) as total_tasks,
coalesce(sum(estimated_hours), 0) as total_hours')
->groupBy('assigned_to')
->get()
->keyBy('assigned_to');

```

Query generada:

```

SELECT
    assigned_to,
    COUNT(*) AS total_tasks,
    COALESCE(SUM(estimated_hours), 0) AS total_hours
FROM tasks
WHERE team_id = '{id}'
    AND status != 'completed'
    AND due_date BETWEEN '{start}' AND '{end}' -- solo si se pasa el rango
GROUP BY assigned_to

```

Una sola query devuelve los totales de TODOS los miembros a la vez. El resultado se indexa por `assigned_to` con `->keyBy('assigned_to')` para acceso en O(1).

Bloque 3 — Calcular carga relativa por persona

```

$stats      = $taskStats->get($member->user_id); // acceso O(1) por UUID
$hours      = $stats ? (float) $stats->total_hours : 0;
$capacity   = (int) ($member->profile->weekly_hours_capacity ?? 40);
$loadPct    = $capacity > 0 ? round($hours / $capacity * 100) : 100;

$workloadLevel = match (true) {
    $loadPct < 40  => 'low',
    $loadPct <= 70 => 'medium',
    default       => 'high',
};

```

Fórmula del porcentaje de carga:

```
workload_pct = ROUND( horas_activas / capacidad_semanal × 100 )
```

Ejemplo:

Michel Reyes: 5h activas / 30h capacidad × 100 = 17% → "low"

Juan Feijoo: 34h activas / 40h capacidad × 100 = 85% → "high"

Fernando: 20h activas / 40h capacidad × 100 = 50% → "medium"

Bloque 4 — Sugerido simple

```
$suggested = $membersData->sortBy('workload_pct')->first();
```

Ordena por porcentaje ascendente y retorna el primero: la persona con menor carga.

Tablas involucradas:

```
teams          ← validación que team_id existe
team_members   ← obtener miembros del equipo
profiles       ← weekly_hours_capacity (capacidad individual)
tasks          ← SUM(estimated_hours) GROUP BY assigned_to
```

4. SUGERENCIAS DE ASIGNACIÓN

Nombre: Workload Suggestions — Algoritmo de Matching **Ruta:**

`backend/app/Http/Controllers/WorkloadController.php` — método `suggestions()`

Endpoint: `GET /api/suggestions?team_id=X`

Este es el algoritmo más complejo: navega 5 tablas y realiza 2 cruces de datos.

Código completo — `suggestions()`

```
public function suggestions(Request $request)
{
    $request->validate([
        'team_id' => 'required|exists:teams,id',
    ]);

    $members = TeamMember::where('team_id', $request->team_id)
        ->with(['profile.skills.practice'])
        ->get();

    $activeProjects = Project::where('team_id', $request->team_id)
        ->where('status', 'active')
        ->withCount(['tasks as pending_tasks_count' => function ($q) {
            $q->where('status', '!=', 'completed');
        }])
        ->with('practice')
        ->get()
        ->filter(fn ($p) => $p->pending_tasks_count > 0)
        ->values();

    // Una query para stats de carga de todos los miembros
    $taskStats = Task::where('team_id', $request->team_id)
        ->where('status', '!=', 'completed')
```

```

        ->selectRow('assigned_to, coalesce(sum(estimated_hours), 0) as
total_hours')
        ->groupBy('assigned_to')
        ->get()
        ->keyBy('assigned_to');

// Una query para todas las tareas pendientes de todos los proyectos
activos
$projectIds      = $activeProjects->pluck('id');
$allPendingTasks = Task::whereIn('project_id', $projectIds)
    ->where('status', '!=', 'completed')
    ->select('id', 'title', 'priority', 'estimated_hours',
'assigned_to', 'project_id')
    ->get()
    ->groupBy('project_id');

$suggestions = $members->map(function ($member) use ($taskStats,
$activeProjects, $allPendingTasks) {
    $stats      = $taskStats->get($member->user_id);
    $hours      = $stats ? (float) $stats->total_hours : 0;
    $capacity   = (int) ($member->profile->weekly_hours_capacity ?? 40);
    $loadPct    = $capacity > 0 ? round($hours / $capacity * 100) : 100;

    if ($loadPct >= 40) {
        return null;
    }

    $userPracticeIds = $member->profile->skills
        ->pluck('practice_id')
        ->filter()
        ->unique()
        ->values();

    $matchingProjects = $activeProjects->filter(
        fn ($p) => $p->practice_id && $userPracticeIds->contains($p-
>practice_id)
    )->values();

    if ($matchingProjects->isEmpty()) {
        return null;
    }

    return [
        'user_id'      => $member->user_id,
        'name'         => $member->profile->first_name . ' ' .
$member->profile->last_name,
        'workload_pct' => $loadPct,
        'assigned_hours' => $hours,
        'capacity'     => $capacity,
        'skills'       => $member->profile->skills->pluck('name')-
>values(),
        'matching_projects' => $matchingProjects->map(fn ($p) => [
            'id'          => $p->id,
            'name'        => $p->name,

```

```

        'practice'           => $p->practice?->name,
        'pending_tasks_count' => $p->pending_tasks_count,
        'pending_tasks'      => $allPendingTasks->get($p->id,
collect()->values(),
        ]->values(),
    ];
    }->filter()->values();

    return response()->json($suggestions);
}

```

Explicación bloque a bloque

Bloque 1 — Cargar miembros con cadena completa de habilidades

```

$members = TeamMember::where('team_id', $request->team_id)
    ->with(['profile.skills.practice'])
    ->get();

```

Eager loading en cascada — 4 tablas en una sola operación:

```
team_members → profiles → user_skills → skills → practices
```

Queries generadas internamente:

```

SELECT * FROM team_members WHERE team_id = '{id}'
SELECT * FROM profiles WHERE id IN (...)
SELECT * FROM user_skills WHERE user_id IN (...)
SELECT * FROM skills WHERE id IN (...)
SELECT * FROM practices WHERE id IN (...)

```

Bloque 2 — Cargar proyectos activos con conteo de tareas pendientes

```

$activeProjects = Project::where('team_id', $request->team_id)
    ->where('status', 'active')
    ->withCount(['tasks as pending_tasks_count' => function ($q) {
        $q->where('status', '!=', 'completed');
    }])
    ->with('practice')
    ->get()
    ->filter(fn ($p) => $p->pending_tasks_count > 0)
    ->values();

```

Query generada:

```

SELECT projects.*,
       (SELECT COUNT(*) FROM tasks
        WHERE tasks.project_id = projects.id
          AND tasks.status != 'completed') AS pending_tasks_count
FROM projects
WHERE team_id = '{id}'
      AND status = 'active'
-- + eager:
SELECT * FROM practices WHERE id IN (practice_id1, practice_id2, ...)

```

Filtra solo proyectos con al menos 1 tarea pendiente (`.filter(fn ($p) => $p->pending_tasks_count > 0)`).

Bloque 3 — Stats de carga (1 query para todos los miembros)

```

$taskStats = Task::where('team_id', $request->team_id)
  ->where('status', '!=', 'completed')
  ->selectRaw('assigned_to, coalesce(sum(estimated_hours), 0) as
total_hours')
  ->groupBy('assigned_to')
  ->get()
  ->keyBy('assigned_to');

```

Query generada:

```

SELECT assigned_to,
       COALESCE(SUM(estimated_hours), 0) AS total_hours
FROM tasks
WHERE team_id = '{id}'
      AND status != 'completed'
GROUP BY assigned_to

```

Bloque 4 — Tareas pendientes de todos los proyectos (1 query)

```

$projectIds      = $activeProjects->pluck('id');
$allPendingTasks = Task::whereIn('project_id', $projectIds)
  ->where('status', '!=', 'completed')
  ->select('id', 'title', 'priority', 'estimated_hours', 'assigned_to',
'project_id')
  ->get()
  ->groupBy('project_id');

```

Query generada:

```
SELECT id, title, priority, estimated_hours, assigned_to, project_id
FROM tasks
WHERE project_id IN (proyecto1_id, proyecto2_id, ...)
AND status != 'completed'
```

El resultado se agrupa por `project_id` para acceso O(1) en el paso siguiente.

Bloque 5 — CRUCE 1: filtrar por carga < 40%

```
$stats      = $taskStats->get($member->user_id);
$hours      = $stats ? (float) $stats->total_hours : 0;
$capacity   = (int) ($member->profile->weekly_hours_capacity ?? 40);
$loadPct    = $capacity > 0 ? round($hours / $capacity * 100) : 100;

if ($loadPct >= 40) {
    return null; // descartado - carga media o alta
}
```

Solo pasan las personas con carga menor al 40% de su capacidad semanal.

Bloque 6 — CRUCE 2: comparar prácticas del usuario vs prácticas de los proyectos

```
$userPracticeIds = $member->profile->skills
->pluck('practice_id') // [uuid_mobile, uuid_ecommerce, ...]
->filter()             // elimina nulls (skills sin práctica)
->unique()             // sin duplicados
->values();

$matchingProjects = $activeProjects->filter(
    fn ($p) => $p->practice_id && $userPracticeIds->contains($p->practice_id)
)->values();
```

Esta comparación en PHP (en memoria, sin query extra):

```
userPracticeIds = [ uuid_ecommerce ]           ← prácticas de las skills del usuario
activeProjects  = [
    { name: "Tienda ABC", practice_id: uuid_ecommerce }, ← COINCIDE ✓
    { name: "App iOS",   practice_id: uuid_mobile }      ← no coincide x
]

matchingProjects = [ "Tienda ABC" ]
```

Ejemplo real con los datos del sistema:

```

Michel Reyes:
  skills: WooCommerce (practice_id = uuid_ecommerce),
         Shopify      (practice_id = uuid_ecommerce),
         Magento      (practice_id = uuid_ecommerce)

userPracticeIds = [ uuid_ecommerce ]

Proyecto "Tienda E-commerce ABC":
  practice_id = uuid_ecommerce
  → uuid_ecommerce ∈ [ uuid_ecommerce ] → MATCH ✓

Proyecto "App iOS para cliente XYZ":
  practice_id = uuid_mobile
  → uuid_mobile ∈ [ uuid_ecommerce ] → NO MATCH ✗

Resultado: Michel Reyes aparece en sugerencias para "Tienda E-commerce
ABC"

```

Tablas involucradas (completo):

```

team_members  ← miembros del equipo
profiles      ← datos personales + weekly_hours_capacity
user_skills   ← junction: qué skills tiene cada usuario
skills        ← nombre de la skill + practice_id
practices     ← tipo de práctica (Mobile, E-commerce, etc.)
projects      ← proyectos activos del equipo + practice_id
tasks        ← conteo pendientes por proyecto + horas activas por miembro

```

5. GESTIÓN DE EQUIPOS Y MIEMBROS

Nombre: Team Controller — Gestión de Miembros **Ruta:**

backend/app/Http/Controllers/TeamController.php **Endpoints:** GET/POST/DELETE
 /api/teams/{id}/members

Código completo — **members()** (listar miembros con carga)

```

public function members(Request $request, $id)
{
    Team::findOrFail($id);

    $teamMembers = TeamMember::where('team_id', $id)->with('profile')-
    >get();

    // Una sola query para todos los miembros en lugar de N queries en el

```

```

loop
  $taskStats = Task::where('team_id', $id)
    ->where('status', '!=', 'completed')
    ->selectRaw('assigned_to, count(*) as total_tasks,
coalesce(sum(estimated_hours), 0) as total_hours')
    ->groupBy('assigned_to')
    ->get()
    ->keyBy('assigned_to');

  $result = $teamMembers->map(function ($member) use ($taskStats) {
    $stats = $taskStats->get($member->user_id);
    $hours = $stats ? (float) $stats->total_hours : 0;
    $tasks = $stats ? (int) $stats->total_tasks : 0;
    $capacity = (int) ($member->profile->weekly_hours_capacity ?? 40);
    $loadPct = $capacity > 0 ? round($hours / $capacity * 100) : 100;

    $workloadLevel = match (true) {
      $loadPct < 40 => 'low',
      $loadPct <= 70 => 'medium',
      default      => 'high',
    };

    return [
      'id'          => $member->user_id,
      'name'        => $member->profile->first_name . ' ' .
$member->profile->last_name,
      'email'       => $member->profile->email,
      'total_tasks' => $tasks,
      'total_hours' => $hours,
      'assigned_hours' => $hours,
      'capacity'    => $capacity,
      'workload_pct' => $loadPct,
      'workload_level' => $workloadLevel,
    ];
  });

  return response()->json($result->values());
}

```

Código completo — `addMember()` (agregar miembro)

```

public function addMember(Request $request, $id)
{
    Team::findOrFail($id);

    $request->validate([
        'profile_id' => 'required|exists:profiles,id',
    ]);

    $exists = TeamMember::where('team_id', $id)
        ->where('user_id', $request->profile_id)

```



```

->exists();

if ($exists) {
  return response()->json([
    'errors' => ['profile_id' => ['El usuario ya es miembro de este
equipo']]
  ], 422);
}

$member = TeamMember::create([
  'team_id' => $id,
  'user_id' => $request->profile_id,
]);

return response()->json($member->load('profile'), 201);
}

```

Código completo — `removeMember()` (quitar miembro)

```

public function removeMember(Request $request, $id, $userId)
{
  Team::findOrFail($id);

  $member = TeamMember::where('team_id', $id)
    ->where('user_id', $userId)
    ->firstOrFail();

  $member->delete();

  return response()->json(['message' => 'Miembro eliminado del equipo']);
}

```

Explicación

`addMember()` — Prevención de duplicados

```

$exists = TeamMember::where('team_id', $id)
  ->where('user_id', $request->profile_id)
  ->exists();

```

Query:

```

SELECT COUNT(*) > 0 FROM team_members
WHERE team_id = '{team_id}'
AND user_id = '{profile_id}'

```

Si ya existe el par (`team_id`, `user_id`) → 422. La tabla además tiene `UNIQUE(team_id, user_id)` a nivel de DB como segunda barrera.

`removeMember()` — Eliminación segura

```
$member = TeamMember::where('team_id', $id)
    ->where('user_id', $userId)
    ->firstOrFail(); // → 404 si no existe la membresía
$member->delete();
```

Si el par no existe devuelve 404 automáticamente via `firstOrFail()`.

Tablas involucradas:

```
teams          ← verificar que el equipo existe (findOrFail)
profiles       ← verificar que el profile_id existe (exists:profiles,id)
team_members   ← insert / select / delete de membresías
tasks          ← aggregation para stats de carga en members()
```

6. GESTIÓN DE USUARIOS Y SKILLS

Nombre: Profile Controller — Gestión de Habilidades **Ruta:**

`backend/app/Http/Controllers/ProfileController.php` **Endpoints:** `GET /api/profiles/{id}/skills`, `POST /api/profiles/{id}/skills`

Código completo — `skills()` (ver skills del usuario)

```
public function skills($id)
{
    $profile = Profile::with('skills.practice')->findOrFail($id);

    return response()->json($profile->skills);
}
```

Query generada:

```
SELECT * FROM profiles WHERE id = '{id}'
-- eager loading en cascada:
SELECT * FROM user_skills WHERE user_id = '{id}'
SELECT * FROM skills WHERE id IN (skill_id1, skill_id2, ...)
SELECT * FROM practices WHERE id IN (practice_id1, practice_id2, ...)
```

Navega: [profiles](#) → [user_skills](#) → [skills](#) → [practices](#)

Código completo — `syncSkills()` (sincronizar skills del usuario)

```
public function syncSkills(Request $request, $id)
{
    $profile = Profile::findOrFail($id);

    $request->validate([
        'skill_ids' => 'present|array',
        'skill_ids.*' => 'exists:skills,id',
    ]);

    $profile->skills()->sync($request->skill_ids);

    return response()->json($profile->skills()->with('practice')->get());
}
```

Explicación

Validación de skills:

```
$request->validate([
    'skill_ids' => 'present|array',    // debe ser un array (puede estar vacío)
    'skill_ids.*' => 'exists:skills,id', // cada UUID debe existir en la tabla skills
]);
```

Por ejemplo: { "skill_ids": ["uuid1", "uuid2", "uuid3"] }.

El método `sync()` — Sincronización diferencial

```
$profile->skills()->sync($request->skill_ids);
```

Eloquent genera internamente estas queries:

```
-- 1. Ver qué skills tiene actualmente el usuario
SELECT skill_id FROM user_skills WHERE user_id = '{profile_id}'
-- resultado actual: [uuid1, uuid2, uuid4]

-- 2. Calcular diferencias con la lista nueva [uuid1, uuid2, uuid3]
-- Para eliminar: uuid4 (estaba, ya no está)
-- Para agregar:  uuid3 (es nueva)
-- Sin cambio:    uuid1, uuid2
```

```
-- 3. DELETE las que ya no están
DELETE FROM user_skills WHERE user_id = '{id}' AND skill_id = 'uuid4'

-- 4. INSERT las nuevas
INSERT INTO user_skills (user_id, skill_id) VALUES ('{id}', 'uuid3')
```

`sync()` hace una sincronización diferencial: no borra y recrea todo, solo hace los cambios mínimos necesarios.

Tablas involucradas:

```
profiles    ← buscar el usuario
skills      ← validar que cada skill_id exista
user_skills ← junction table que se sincroniza (DELETE + INSERT selectivos)
practices   ← incluida en la respuesta via eager loading
```

RESUMEN EJECUTIVO — NAVEGACIÓN ENTRE TABLAS

Funcionalidad	Tablas navegadas en orden	Cruces de datos
Autenticación	ninguna (HTTP remoto)	—
Crear tarea	teams + profiles + projects → team_members	1 cruce: assigned_to ∈ team_members
Carga de trabajo	team_members → profiles → tasks	1 agregación: GROUP BY assigned_to
Sugerencias	team_members → profiles → user_skills → skills → practices + projects → tasks	2 cruces: carga < 40% + skills.practice_id = projects.practice_id
Miembros equipo	team_members → profiles + tasks	1 agregación: GROUP BY assigned_to
Sync skills	profiles ↔ user_skills ↔ skills	sync diferencial

El dato que conecta todo el sistema

```
practices.id
├─> skills.practice_id    "React Native pertenece a la práctica Mobile"
└─> projects.practice_id "App iOS es un proyecto de práctica Mobile"

Comparación en suggestions():
```

```
skills.practice_id == projects.practice_id  
"Lo que sabe hacer" == "Lo que el proyecto necesita"
```